

RepoMind

Retrieval Engineering Documentation

Codebase RAG • Issues • Fixes • Current Limitations • Future Improvements

Purpose

This document records the major retrieval and code-understanding problems encountered while building RepoMind, the fixes that were implemented, the results observed during testing, and the improvements deliberately left for future versions. It is intended to serve as project documentation and as a technical explanation of why the current architecture is a solid v1 rather than an attempt to reproduce a production coding assistant.

Current project level: Intermediate/advanced student-level codebase RAG with code-aware chunking, hybrid retrieval, RRF, and cross-encoder reranking.

1. Current RepoMind Architecture

The current end-to-end pipeline is modular and separates ingestion, code-aware chunking, indexing, retrieval, reranking, context construction, and generation.

```
GitHub repository
↓
Repository loader / scanner
↓
Language-aware file filtering
↓
Tree-sitter AST parsing
↓
Semantic + size-aware code chunking
↓
Embeddings
↓
ChromaDB
+
Persistent BM25 index
↓
Vector retrieval + lexical retrieval
↓
Reciprocal Rank Fusion (RRF)
↓
BGE cross-encoder reranking
↓
Top-K context
↓
Groq LLM
↓
Grounded answer + source metadata
```

The separation makes it possible to diagnose individual stages instead of treating the RAG system as a black box.

2. Why RepoMind Is More Than Basic Document RAG

A basic document chatbot commonly splits arbitrary text, embeds the chunks, performs similarity search, and sends the retrieved text to an LLM. RepoMind treats source code as structured data.

- **AST-aware chunking:** Tree-sitter identifies functions, classes, methods, interfaces, types, enums, structs, and other language-specific units.
- **Language-aware behavior:** JavaScript/TypeScript, Python, Java, C/C++, Go, HTML, CSS, JSON, and Markdown use structure-specific chunking.
- **Nested-function protection:** Arrow functions and function expressions assigned to variables are promoted to their enclosing declaration.
- **Hybrid retrieval:** vector search and BM25 are combined because code queries often contain exact identifiers.
- **RRF:** independent ranked lists are fused without directly adding incompatible vector and BM25 scores.
- **Cross-encoder reranking:** candidate chunks are evaluated with the query and candidate together before final context construction.

3. Issue — Naive AST Chunking Lost Code Context

Problem: The first Tree-sitter implementation selected nested *arrow_function* or *function* nodes directly. For code such as the following, callbacks could become independent chunks:

```
const httpLogger = pinoHttp({
  genReqId: (req) => ...,
  customLogLevel: (_req, res) => ...,
  ...
})
```

```
});
```

That produced chunks corresponding to callback bodies rather than the meaningful repository symbol *httpLogger*. Similar behavior occurred with functions assigned to variables.

Why it mattered: A query for the surrounding identifier could fail because the declaration name was absent from the retrieved chunk.

4. Fix — Promote Nested Functions to Their Variable Declaration

The AST selection logic was changed so that when an arrow function or function expression is inside a variable declarator, the enclosing lexical declaration becomes the chunk boundary.

```
arrow_function / function
↓
enclosing_variable_declarator
↓
enclosing_lexical_declaration
↓
one semantic chunk
```

Nodes are deduplicated so multiple nested callbacks do not create duplicate declaration chunks.

Observed result: The request logger became one meaningful *httpLogger* chunk containing its nested callbacks. In the tested express-typescript repository, the chunk count dropped from 152 to 140 after this AST fix.

5. Issue — Embeddings Initially Ignored Useful Metadata

The first indexing implementation embedded only *chunk.content*. File path, language, type, symbol, and line information were stored as metadata but did not participate in the semantic representation used for vector similarity.

Observed effect: File-oriented queries could retrieve README, tests, or unrelated source instead of strongly identifying the requested implementation file.

6. Fix — Include Retrieval-Relevant Metadata in Embedding Text

The indexed text was changed to include file path, language, type, and symbol before the source content:

```
File: src/common/middleware/requestLogger.ts
Language: typescript
Type: variable
Symbol: httpLogger

const httpLogger = pinoHttp({
  ...
});
```

The same information remains available as Chroma metadata. Embedding it as text gives vector retrieval explicit information about file identity and symbols.

Observed result: Exact file-oriented vector queries improved substantially and began returning requestLogger chunks near the top instead of unrelated README/test material.

7. Issue — One Retrieval Signal Is Not Enough for Code

Code search has both semantic and exact-match requirements. “How is request logging implemented?” benefits from semantic similarity, while “Where is addRequestId defined?” benefits strongly from exact lexical matching.

8. Fix — Persistent BM25 + Vector Retrieval

RepoMind added a persistent BM25 index using *wink-bm25-text-search*. The index is built during repository indexing, exported to disk, and loaded for later queries instead of being rebuilt for every prompt.

```
Vector search ████████
████ Hybrid retrieval
BM25 search ██████████
```

BM25 fields weight file paths and symbols more strongly than ordinary content. This is useful for exact identifiers. BM25 remains lexical and does not inherently understand relationships between different identifiers.

9. Issue — Vector and BM25 Scores Are Not Directly Comparable

Vector similarity and BM25 scores have different meanings and scales, so simply adding their raw scores is not a principled fusion strategy.

10. Fix — Reciprocal Rank Fusion

RRF combines ranked positions rather than raw scores:

```
RRF(chunk) = 1 / (k + vectorRank)
+ 1 / (k + keywordRank)

Current k = 60
```

A chunk appearing near the top of both retrieval lists receives contributions from both. RRF therefore creates a robust candidate set without requiring score calibration.

11. Issue — Hybrid Retrieval Still Produces False Positives

Hybrid retrieval improved candidate discovery, but the candidate pool can still contain README sections, tests, configuration, or merely related source. The first stage only needs to produce a manageable candidate set.

12. Fix — BGE Cross-Encoder Reranking

RepoMind added a BGE cross-encoder reranker. Unlike embeddings, which encode query and document separately, the reranker receives the query and candidate together and produces a relevance score.

```
Query + candidate chunk
↓
cross-encoder model
↓
relevance score
↓
sort candidates
↓
Top-K
```

The reranker is intentionally placed after hybrid retrieval. It improves precision among candidates but cannot recover a chunk that never entered the candidate pool.

Observed result: For “Where is `app.use()` used?”, reranking moved the relevant `src/server.ts` result above less relevant candidates.

13. Current Retrieval Pipeline

```
User query
↓
Vector search (Top 15)
+
BM25 search (Top 15)
↓
RRF candidate fusion (Top ~10)
```

```
↓
BGE cross-encoder reranker
↓
Top 5
↓
Context builder
↓
Groq LLM
```

14. Remaining Issue — Broad Repository-Level Queries

The main unresolved limitation is repository-level relationship discovery. Some questions require the system to discover several entities and then follow relationships between registration sites, imports, exports, definitions, and implementation symbols.

Representative query: “List all the middlewares and their implementation.”

```
server.ts
■
■■■ rateLimiter
■■■ requestLogger
■■■ errorHandler
■
■■■ unexpectedRequest
■■■ addErrorToRequestLog
```

Implementation chunks can expose different symbols. For example, the request logger implementation contains symbols such as *httpLogger*, *addRequestId*, and *captureResponseBody*, while the registration site refers to *requestLogger*. Pure lexical retrieval therefore cannot reliably connect the two.

15. Current Capabilities vs. Remaining Limitations

Works well for	Still limited for
Exact symbol and identifier lookup	Cross-file relationship traversal
Implementation-oriented questions	Broad “list every X and its implementation” questions
Conceptual questions with relevant source chunks	Following public names to differently named internal symbols
File/path-oriented retrieval	Repository-wide dependency/definition discovery
Hybrid semantic + lexical retrieval	Automatically constructing a complete code graph

16. Important Diagnostic Lesson — Reranking Cannot Recover Missing Candidates

If a relevant implementation chunk never enters the first-stage candidate pool, the reranker cannot select it.

```
Missing from first-stage retrieval
↓
never reaches RRF
↓
never reaches reranker
↓
cannot appear
in final context
```

This means retrieval debugging should first examine recall and candidate coverage before changing the LLM or reranker.

17. Future Improvement — Candidate Expansion

A possible next step is to use initial results to discover important identifiers and then perform additional targeted searches. For example, if `server.ts` reveals `rateLimiter`, `requestLogger`, and `errorHandler`, those terms can be searched individually before final RRF and reranking.

```
Query
↓
Initial vector + BM25 retrieval
↓
Discover identifiers / symbols
↓
Additional targeted searches
↓
Expanded candidate pool
↓
RRF
↓
Reranker
↓
Top-K
```

This was deliberately not implemented in v1. It adds another retrieval stage and should be introduced only if broader repository coverage becomes a project requirement.

18. Future Improvement — Reference-Aware Retrieval

A more powerful approach is to extract imports, exports, definitions, calls, and references so retrieval can follow relationships rather than relying only on textual similarity.

```
server.ts
↓ references
requestLogger
↓ definition / import relationship
requestLogger.ts
↓ contains
httpLogger / addRequestId / captureResponseBody
```

This would move RepoMind toward repository-aware or graph-aware retrieval. It is substantially more complex than ordinary RAG and is better treated as a future version.

19. Future Improvement — OpenSearch

OpenSearch could consolidate parts of the current retrieval infrastructure by providing lexical search, vector search, metadata filtering, and hybrid retrieval in one system. It could reduce custom BM25/RRF plumbing.

However: changing to OpenSearch alone does not solve the unresolved cross-file relationship problem. The missing capability is repository-aware discovery, not merely a different search backend. OpenSearch should therefore be considered an infrastructure simplification, not the direct fix for the current limitation.

20. Future Improvement — GitHub API Ingestion

The repository acquisition layer can later be changed from Git cloning to GitHub API-based ingestion while keeping the downstream `CodeFile` → semantic chunk → indexing concepts intact.

```
Current:
GitHub URL
↓
git clone
↓
local repository
↓
scanner
```

```
Possible future:
GitHub URL
↓
GitHub API
↓
repository tree + file contents
↓
scanner / CodeFile[]
```

This is an ingestion improvement and does not directly solve retrieval relationships, so it should remain independent from retrieval work.

21. Improvements Intentionally Deferred in v1

- Repository-wide code graph construction
- Identifier/reference expansion
- Query rewriting or multi-query retrieval
- HyDE
- OpenSearch migration
- Additional embedding-model experiments after the Voyage rate-limit issue
- Copilot-style agentic code modification
- Complex dependency analysis
- Premature optimization of BM25 mapping or chunk lookup

These are possible future directions, not missing requirements. The current goal was to make the complete RAG pipeline work, understand each stage, test it, and establish a stable v1 before adding additional complexity.

22. Recommended v1 Boundary

```
RepoMind v1
█
██████████████████████████████████████████████████
↓ ↓ ↓
Ingestion Retrieval Generation
█ █ █
repo scanner Vector + BM25 Groq
file filtering RRF context
AST chunking BGE reranker citations
█ █
████████ stable and tested ████████
```

The current implementation is sufficient to demonstrate a serious codebase RAG architecture. The remaining weaknesses should be documented rather than hidden.

23. Key Engineering Lessons

- 1. Chunking determines retrieval quality.** If semantic boundaries are wrong, later retrieval stages cannot fully compensate.
- 2. Code needs multiple retrieval signals.** Exact identifiers and conceptual meaning are both important.
- 3. RRF solves score incompatibility, not semantic understanding.** It is a practical bridge between retrieval systems
- 4. Reranking improves precision, not recall.** A reranker can reorder candidates but cannot recover missing documents.

5. Metadata can matter semantically. Including file path and symbol information in embedding text improved file-oriented retrieval.

6. Code RAG eventually becomes a relationship problem. Cross-file references and differing public/internal names are difficult for pure similarity search.

7. Perfection is not the objective. A well-tested v1 with known limitations is more useful than an unfinished attempt to reproduce a production-scale coding assistant.

24. Final Status

RepoMind v1 is complete enough to freeze. It has a full ingestion-to-generation pipeline, code-aware semantic chunking, persistent lexical retrieval, semantic retrieval, rank fusion, reranking, and end-to-end LLM generation. The remaining weakness is repository-level relationship discovery for broad multi-entity questions.

Future work can target that limitation through candidate expansion, reference-aware retrieval, or a code graph. OpenSearch and GitHub API can be explored independently when there is a concrete reason to change the underlying infrastructure.

Project Positioning

RepoMind should be presented as a **codebase RAG / hybrid retrieval project**, not as a Copilot replacement. Its strongest differentiator is code-aware semantic chunking combined with lexical retrieval, semantic retrieval, RRF, and cross-encoder reranking.

Recommended resume description: Built a codebase RAG system with AST-based semantic chunking, hybrid vector/BM25 retrieval, Reciprocal Rank Fusion, and BGE cross-encoder reranking for repository-aware question answering.